PODFlow AI — Technical Design Document

GitHub URL: ADD YOUR PUBLIC GITHUB REPOSITORY URL BEFORE SUBMISSION

Live application: Not deployed yet

Document status: Prototype design and implementation notes

1. Executive summary

PODFlow AI is a prototype for capturing proof-of-delivery (POD) documents, flagging uncertain submissions for billing review, and preparing verified trips for a Tally-compatible CSV handoff. The implementation focuses on the supplied core workflow: driver upload, advisory document checks, human review, billing queue and missing-POD overview.

This is a demonstration prototype, not a production-ready transport platform.

2. Problem and goals

The client reports that paper PODs can take 10–20 days to return, delaying invoices. The brief estimates that reducing POD-to-invoice time from 18 to 4 days could bring billing forward by 14 days and release approximately ₹3.24 crore in working capital, using the client's stated assumptions.

Prototype goals:

- Make a trip's POD upload simple.
- Validate and compress image uploads to at most 300 KB.
- Extract visible text where OCR is available, while clearly flagging uncertainty.
- Keep the original image as the source of truth.
- Require a billing executive to review before a trip becomes eligible for export.
- Prevent more than two retake requests.
- Provide seeded sample trips so the demo is not empty.

3. Scope and assumptions

Implemented in this prototype

- Driver and billing executive demo sign-in.
- Role-protected API endpoints.
- Seeded fictional trip records with LR and e-way bill references.
- POD image upload, validation and compression.
- Optional Tesseract OCR text extraction.
- Human review queue with original POD image display.
- Two-retake request limit.
- Verified-trip billing queue and CSV export.
- Missing-POD and status overview.
- Automated tests for key workflow rules.

Explicit assumptions

1. The three trips and additional seeded records are fictional examples based on the supplied sample.

2. Demo accounts represent the two required roles; they are not individual employee accounts.

3. A verified status is assigned only by a billing executive, not by OCR.

4. Tally export means a CSV handoff template, not direct Tally API integration.

5. The local SQLite database and local upload directory are acceptable for a single-machine demonstration.

6. No delivery or invoice timestamps were supplied, so cycle-time KPIs are not calculated.

7. The prototype does not infer GPS coordinates from a browser or image.

4. Architecture and technology choices

• Layer • Technology • Reason •

• --- • --- • --- •

• User interface • Streamlit • Fast Python UI for a small role-based prototype •

• API • FastAPI • Clear REST endpoints, request validation and API documentation •

• Database • SQLite • Lightweight local persistence for a solo demonstration •

• Image processing • Pillow • Decode, orient, resize and compress uploads •

• Text extraction • Tesseract OCR via pytesseract • Optional local OCR without sending documents to a third-party service •

• Tests • pytest + FastAPI TestClient • Verify authentication and critical workflow rules •

Request flow

1. User signs in through the Streamlit UI.

2. FastAPI validates demo credentials and returns a signed bearer token.

3. Streamlit sends the token with protected API requests.

4. Driver uploads a POD image for a known trip.

5. API validates the image, compresses it under 300 KB and saves the original processed image.

6. OCR attempts to extract text and compare the expected LR number.

7. The trip is marked `Needs review`; it is not auto-approved.

8. Billing executive opens the original image, checks it and records a decision.

9. Only `Verified` trips appear in the billing export queue.

5. Data model

The trips table stores:

• `trip_id` — unique trip identifier

• `client`, `vehicle`, `destination`, `freight`

• `status` — Missing POD, Needs review, Retake requested or Verified

• `remarks` — OCR flags or reviewer notes

• `lr`, `eway` — lorry receipt and e-way bill references

• `branch`

• `pod_path` — stored image filename

• `invoice_status` — Not ready, Ready or Exported

• `retake_count` — retake requests used, capped at two

• `uploaded_at`, `verified_at`, `exported_at`, `delivery_date`

The schema is initialized and seeded on first API startup. Additive migrations create new workflow fields for an earlier starter database.

6. Main workflows

Driver POD capture

The driver signs in, selects a trip and uploads a JPG, PNG or WEBP image. The API rejects empty, invalid or over-10-MB source uploads. The image is oriented, resized and compressed; upload is rejected if the result cannot be brought under 300 KB.

OCR check and retake handling

Tesseract attempts text extraction and checks for the expected LR string. Date and quantity-related text patterns are advisory hints only. The system cannot reliably confirm a consignee stamp, signature, quantity, or shortage/damage remarks from OCR. These are explicitly assigned to human review.

The billing executive can request a retake. The API enforces a maximum of two retake requests. A third request is rejected with HTTP 409.

Manual review

The billing executive sees the trip metadata, original POD image, OCR flags and prior retake count. The reviewer may mark the trip Verified, Needs review or Retake requested. Only Verified changes invoice status to Ready.

Billing export

The billing executive downloads a CSV containing trip, client, LR, e-way bill, vehicle, destination, freight and POD filename. The executive can mark a verified trip Exported. This is a handoff file; no invoice is automatically sent and no direct Tally connection exists.

Missing-POD overview

The dashboard summarizes trip counts by status and shows missing PODs by branch. It does not show POD-to-invoice duration because the sample data lacks trustworthy timestamps.

7. AI / automation design and failure handling

The current implementation uses local OCR, not a trained vision model. OCR is used only as an assistive signal:

• OCR unavailable: flag for manual review.

• No readable text: flag for retake or manual review.

• LR mismatch: flag for manual review.

• Date or quantity keywords not detected: flag for manual review.

• Stamp, signature and shortage/damage notes: always require visual human confirmation.

The uploaded image remains the source of truth. The backend never auto-approves a POD. This is a deliberate safety rule because false verification could result in a rejected invoice or disputed freight claim.

8. UI and design decisions

• Restrained navy, white and pale-grey palette to support focus.

• Role-specific navigation so drivers see capture and trip status, while billing users see operations, review and billing.

• Clear status labels and metric cards for the operations view.

• Image preview before upload and original image in review.

• Short instructions and a prominent upload action for drivers.

• Error messages for failed uploads, unavailable API and blocked actions.

The Streamlit interface is responsive to available browser width, but a dedicated low-literacy mobile application with Hindi/Marathi voice prompts is future work.

9. Security and privacy

Implemented prototype controls:

• Bearer token required for protected API routes.

• Driver-only POD upload.

• Billing-only review and export.

• HMAC signature validation for demo tokens.

• Upload size/type checks and safe generated filenames.

• No API keys or real personal data included in the source package.

• Fictional sample records only.

Production requirements:

• Individual employee accounts, password hashing and token expiry/revocation.

• Strong secret management, HTTPS, rate limiting and audit logs.

• Private object storage, retention/deletion policies and backups.

• Malware scanning and content-type verification.

• Formal DPDP Act 2023 assessment and notice/consent/retention controls for consignee personal data.

• Role/branch-level authorization review.

10. Tests and validation

The automated test suite covers:

• Health endpoint and seeded trip data.

• Authentication requirement.

• Driver cannot perform billing review.

• Billing executive can verify a trip and access export.

• Upload compression under 300 KB.

• Upload is never auto-approved.

• Retake request cap of two.

• Driver cannot access billing export.

Local test result at preparation time: 9 tests passed. Run pytest -q from the project root to reproduce.

11. What was cut and next steps

Not implemented in this prototype:

• Durable offline capture and background retry/sync.

- Native driver app, Hindi/Marathi voice prompts and SMS upload link for hired drivers.
- Actual vision AI model for stamp, signature, quantity, date and damage/shortage verification.
- GPS capture and timestamp attestation at consignee gate.
- Direct Tally integration and invoice PDF generation with embedded POD images.
- Client-facing POD portal.
- Delivery/invoice cycle-time analytics.
- Production hosting, PostgreSQL and durable private object storage.

Recommended next work, in priority order:

1. Build a mobile-first PWA/native capture workflow with encrypted local queue, offline persistence, retry and sync conflict handling.
2. Add a validated vision model and confidence thresholds using consented, representative POD samples; keep human approval mandatory.
3. Add GPS/time capture with spoofing and permission limitations documented.
4. Add SMS one-time upload links with expiry and trip-bound authorization.
5. Add invoice generation, attachment bundling and tested Tally import mapping.
6. Add audit trail, individual users, production storage and DPDP controls.
7. Deploy the API and UI with persistent storage and monitoring.

12. Known limitations

The current product is a local prototype. It demonstrates a useful subset of the workflow but does not meet every item in the full brief. In particular, offline capture, actual vision AI, GPS stamping, SMS links and direct Tally integration remain unimplemented. These gaps are stated explicitly rather than represented as completed features.